



Recursion, Searching, Sorting and Big O

Chapter 11

11.1 Introduction

- Recursive functions (or methods) call themselves, either directly or indirectly through other functions (or methods)
 - Can help solve problems more naturally when an iterative solution is not apparent
- We'll look at searching and sorting arrays and other sequences
 - Fascinating problems, because no matter what algorithms you use, the final result is the same
 - Choose algorithms that perform "the best"—most likely, the ones that run the fastest or use the least memory
- For big data applications, you'll want to choose algorithms that are easy to parallelize
 - Put lots of processors to work simultaneously
- Intimate relationship between algorithm design and performance
- Simplest and most apparent algorithms often perform poorly and that developing more sophisticated algorithms can lead to superior performance
- Big O notation concisely classifies algorithms by how hard they have to work to get the job done
 - Helps you compare the efficiency of algorithms.
- Optional section: Develop an animated visualization of the selection sort so you can see it "in action."
 - Great for understanding how algorithms work

11.2 Factorials

- Factorial of a positive integer n is written $n!$ and pronounced “n factorial”
- $n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$
- $1!$ is 1
- $0!$ is defined to be 1
- $5!$ is the product $5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is 120

Iterative Factorial Approach

- Calculate $5!$

```
In [1]: factorial = 1
```

```
In [2]: for number in range(5, 0, -1):  
        factorial *= number
```

```
In [3]: factorial
```

```
Out[3]: 120
```

11.3 Recursive Factorial Example

Recursive Factorial Approach

- Recursive factorial representation can be written as:

$$n! = n \cdot (n - 1)!$$

- 5! is equal to 5 · 4!, as in:

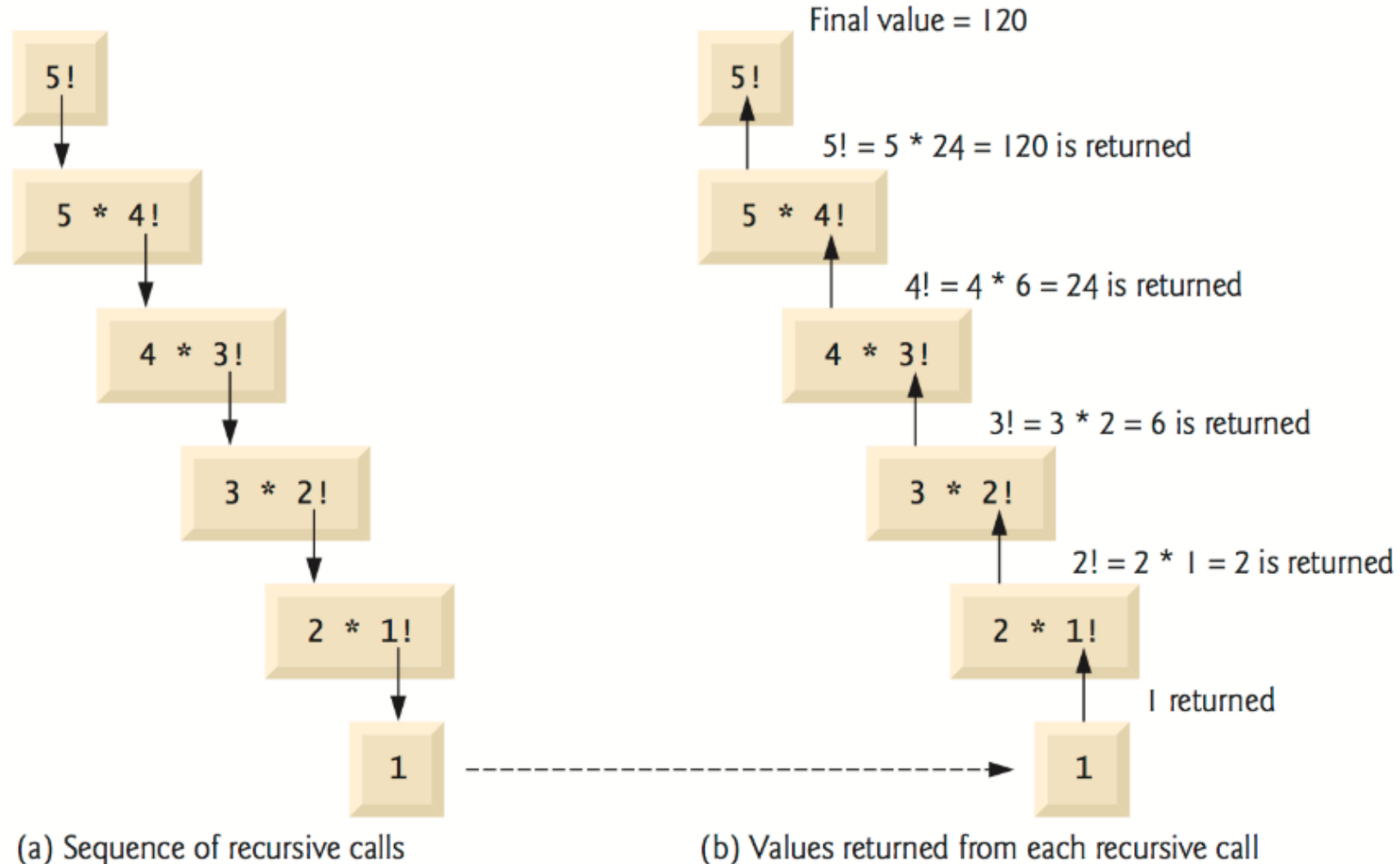
$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

$$5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1)$$

$$5! = 5 \cdot (4!)$$

Visualizing Recursion

- Evaluation of $5!$ would proceed as shown below
- Left column shows how the succession of recursive calls proceeds until $1!$ (the base case) is evaluated to be 1
- Right column shows from bottom to top the values returned from each recursive call to its caller until the final value is calculated and returned



Implementing a Recursive Factorial Function

- Recursive function `factorial` first determines whether the terminating condition `number <= 1` is True
- If True (the base case), `factorial` returns 1
 - No further recursion is necessary
- If number is greater than 1, the second `return` statement expresses the problem as the product of `number` and a recursive call to `factorial` that evaluates `factorial(number - 1)`
 - Slightly smaller problem than the original calculation, `factorial(number)`
- Function `factorial` must receive a nonnegative argument

```
In [1]: def factorial(number):  
        """Return factorial of number."""  
        if number <= 1:  
            return 1  
        return number * factorial(number - 1) # recursive call
```

- Call `factorial` for the values 0-10
- Factorial values grow quickly
- Python does not limit the size of an integer, unlike many other programming languages

```
In [2]: for i in range(11):  
        print(f'{i}! = {factorial(i)}')
```

```
0! = 1  
1! = 1  
2! = 2  
3! = 6  
4! = 24  
5! = 120  
6! = 720  
7! = 5040  
8! = 40320  
9! = 362880  
10! = 3628800
```

Indirect Recursion

- A recursive function may call another function, which may, in turn, make a call back to the recursive function
- Known as an indirect recursive call or indirect recursion

Stack Overflow and Infinite Recursion

- Amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the function-call stack
- If more recursive function calls occur than can have their activation records stored on the stack, a fatal error known as stack overflow occurs
- Typically is the result of infinite recursion
 - Caused by omitting the base case or
 - writing the recursion step incorrectly so that it does not converge on the base case

11.4 Recursive Fibonacci Series Example

- Fibonacci series, begins with 0 and 1 and has the property that each subsequent Fibonacci number is the sum of the previous two

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

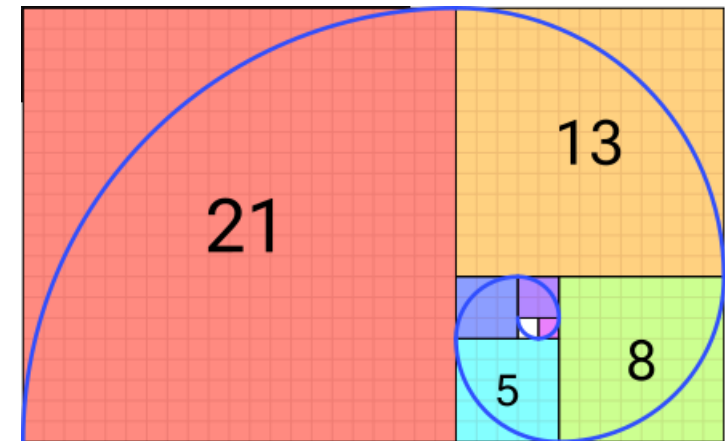
- This series occurs in nature and describes a form of spiral
- Ratio of successive Fibonacci numbers converges on a constant value of 1.618...
 - Called the golden ratio or the golden mean
- Humans tend to find the golden mean aesthetically pleasing
 - Architects often design windows, rooms and buildings whose length and width are in the ratio of the golden mean
 - Postcards often are designed with a golden-mean length-to-width ratio
- Defined recursively as follows:

`fibonacci(0)` is defined to be 0

`fibonacci(1)` is defined to be 1

`fibonacci(n) = fibonacci(n - 1) + fibonacci(n - 2)`

- Two base cases
 - `fibonacci(0)` is 0, and
 - `fibonacci(1)` is 1.



Function `fibonacci`

- Initial call to function `fibonacci` is not a recursive call
- All subsequent calls from function `fibonacci`'s block are recursive
- Each call immediately tests for the base cases
 - n equal to 0 or n equal to 1
 - If so, returns n , because `fibonacci(0)` is 0 and `fibonacci(1)` is 1
- n greater than 1 generates two recursive calls for slightly smaller problems than the original call

```
In [1]: def fibonacci(n):  
        if n in (0, 1): # base cases  
            return n  
        else:  
            return fibonacci(n - 1) + fibonacci(n - 2)
```

Testing Function fibonacci

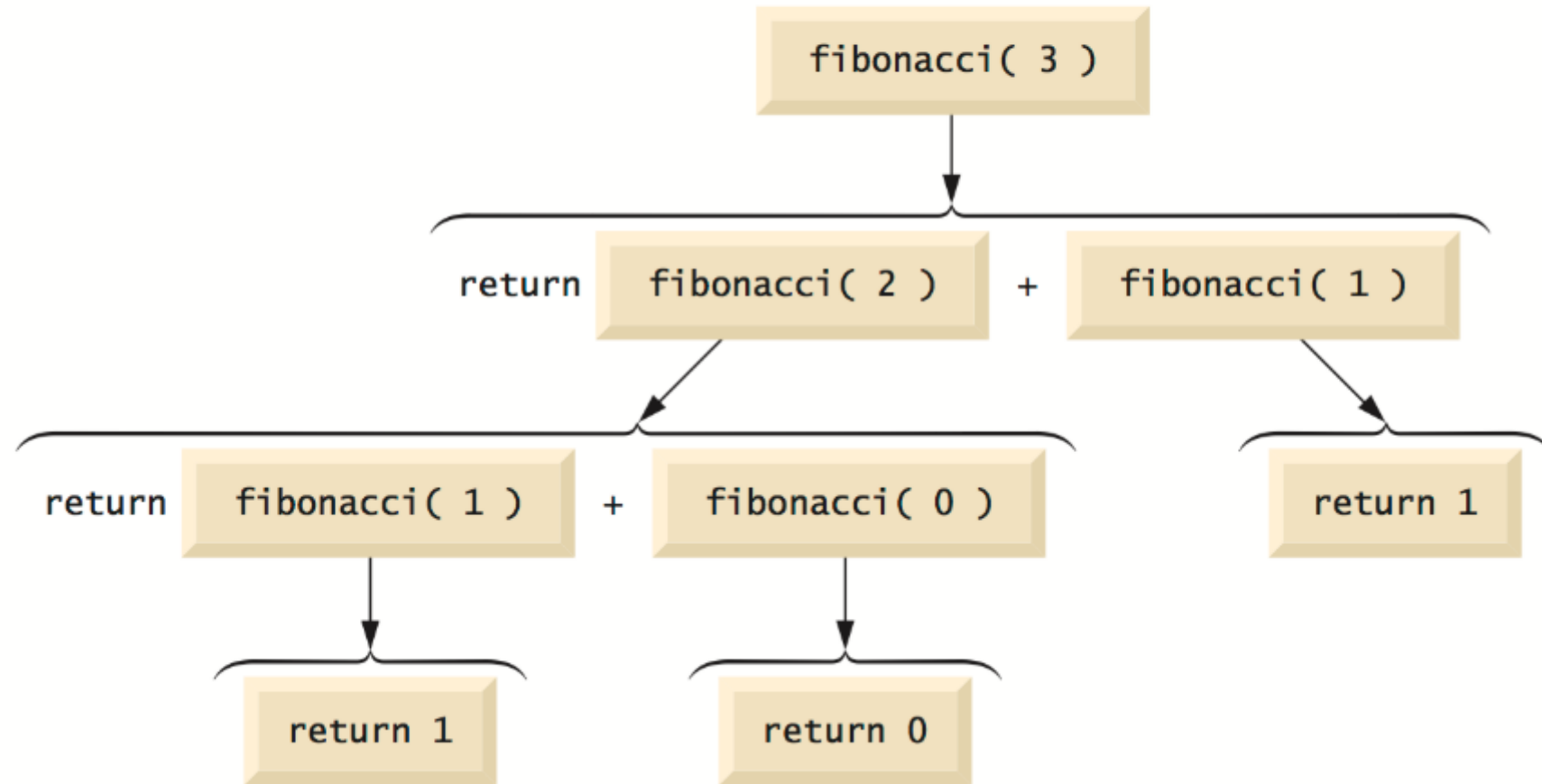
```
In [2]: for n in range(41):  
        print(f'Fibonacci({n}) = {fibonacci(n)}')
```

```
Fibonacci(0) = 0  
Fibonacci(1) = 1  
Fibonacci(2) = 1  
Fibonacci(3) = 2  
Fibonacci(4) = 3  
Fibonacci(5) = 5  
Fibonacci(6) = 8  
Fibonacci(7) = 13  
Fibonacci(8) = 21  
Fibonacci(9) = 34  
Fibonacci(10) = 55  
Fibonacci(11) = 89  
Fibonacci(12) = 144  
Fibonacci(13) = 233  
Fibonacci(14) = 377  
Fibonacci(15) = 610  
Fibonacci(16) = 987  
Fibonacci(17) = 1597  
Fibonacci(18) = 2584  
Fibonacci(19) = 4181  
Fibonacci(20) = 6765
```

```
Fibonacci(21) = 10946  
Fibonacci(22) = 17711  
Fibonacci(23) = 28657  
Fibonacci(24) = 46368  
Fibonacci(25) = 75025  
Fibonacci(26) = 121393  
Fibonacci(27) = 196418  
Fibonacci(28) = 317811  
Fibonacci(29) = 514229  
Fibonacci(30) = 832040  
Fibonacci(31) = 1346269  
Fibonacci(32) = 2178309  
Fibonacci(33) = 3524578  
Fibonacci(34) = 5702887  
Fibonacci(35) = 9227465  
Fibonacci(36) = 14930352  
Fibonacci(37) = 24157817  
Fibonacci(38) = 39088169  
Fibonacci(39) = 63245986  
Fibonacci(40) = 102334155
```

Analyzing the Calls to Function `fibonacci`

- Evaluation of `fibonacci(3)`



Complexity Issues

- Caution about recursive programs
- Each non-base-case invocation of `fibonacci` results in two more recursive calls
- This set of recursive calls rapidly gets out of hand
- `fibonacci(20)` requires 21,891 calls
- `fibonacci(30)` requires 2,692,537 calls!
- Each consecutive Fibonacci number you calculate results in a substantial increase in calculation time and in the number of calls
- `fibonacci(31)` requires 4,356,617 calls
- `fibonacci(32)` requires 7,049,155 calls!
- Problems of this nature can humble even the world's most powerful computers
- Avoid Fibonacci-style recursive programs, because they result in an exponential “explosion” of function calls

11.5 Recursion vs. Iteration

- Both iteration and recursion are based on a control statement
 - Iteration uses an iteration statement
 - Recursion uses a selection statement
- Both involve iteration
 - Iteration explicitly uses an iteration statement
 - Recursion achieves iteration through repeated function calls
- Both involve a termination test
 - Iteration terminates when the loop-continuation condition fails
 - Recursion terminates when a base case is reached
- Both gradually approach termination
- Both can occur infinitely
 - An infinite loop occurs with iteration if the loop-continuation test never becomes false
 - Infinite recursion occurs if the recursion step does not reduce the problem in a manner that converges on the base case, or if the base case is mistakenly not tested

Negatives of Recursion

- Recursion repeatedly invokes the mechanism, and consequently the overhead, of function calls
- Can be expensive in terms of both processor time and memory space
- Each recursive call causes another copy of the function's variables to be stored in the stack frame
 - Can consume considerable memory space
- Iteration avoids these repeated function calls and extra memory assignments
- However, for some algorithms that are easily expressed and understood with recursion, iterative solutions are not readily apparent

11.6 Searching and Sorting

- Searching data involves determining whether a value (referred to as the search key) is present in the data and, if so, finding its location
 - Two popular search algorithms are the simple linear search and the faster but more complex binary search
- Sorting places data in ascending or descending order, based on one or more sort keys
- We selection sort and insertion sort, along with the more efficient but more complex merge sort

11.7 Linear Search

Linear Search Algorithm

- Searches each element in an array sequentially
- If the search key does not match an element in the array, the algorithm informs the user that the search key is not present
- If the search key is in the array, the algorithm tests each element until it finds one that matches the search key and returns the index of that element

Linear Search Algorithm (cont.)

- Consider

35 73 90 65 23 86 43 81 34 58

- Searching for 86
 - Algorithm first checks whether 35 matches the search key
 - It does not, so the algorithm checks whether 73 matches the search key
 - Continues moving through the array sequentially, testing 90, then 65, then 23
 - When the program tests 86, which matches the search key, the program returns the index 5
 - If, after checking every array element, the program determines that the search key does not match any element in the array, it returns a sentinel value (e.g., -1)

Linear Search Implementation

- `linear_search` linear searches an array of integers
- Receives the array to search (`data`) and the `search_key`
- The `for` loop iterates through `data`'s elements and compares each with `search_key`
- Returns the index of the matching element or -1

```
In [1]: def linear_search(data, search_key):  
        for index, value in enumerate(data):  
            if value == search_key:  
                return index  
        return -1
```

```
In [2]: import numpy as np
```

```
In [3]: np.random.seed(11)
```

```
In [4]: values = np.random.randint(10, 91, 10)
```

```
In [5]: values
```

```
Out[5]: array([35, 73, 90, 65, 23, 86, 43, 81, 34, 58])
```

```
In [6]: linear_search(values, 23)
```

```
Out[6]: 4
```

```
In [7]: linear_search(values, 61)
```

```
Out[7]: -1
```

```
In [8]: linear_search(values, 34)
```

```
Out[8]: 8
```

11.8 Efficiency of Algorithms: Big O

- Searching algorithms all accomplish the same goal—finding an element matching a given search key if such an element exists
- Several things differentiate search algorithms from one another
- The major difference is the amount of effort they require to complete the search
- Described with Big O notation, which indicates how hard an algorithm may have to work to solve a problem
- Depends mainly on how many data elements there are
- We use Big O to describe the worst-case run times for various searching and sorting algorithms

O(1) Algorithms

- Suppose an algorithm is designed to test whether the first element of an array is equal to the second
- If the array has 10 elements, this algorithm requires one comparison
- If the array has 1000 elements, it still requires one comparison
- Algorithm is completely independent of the number of elements in the array
- Said to have a constant run time, which is represented in Big O notation as **$O(1)$**
- Pronounced as “order one”
- An algorithm that’s **$O(1)$** does not necessarily require only one comparison
 - Means that the number of comparisons is constant—it does not grow as the size of the array increases

O(n) Algorithms

- An algorithm that tests whether the first array element is equal to any of the other array elements requires at most $n - 1$ comparisons, where n is the number of array elements
- If the array has 10 elements, this algorithm requires up to nine comparisons
- If the array has 1000 elements, it requires up to 999 comparisons
- As n grows larger, the n part of $n - 1$ “dominates,” and subtracting 1 becomes inconsequential
- Big O is designed to highlight dominant terms and ignore terms that become unimportant
- An algorithm that requires a total of $n - 1$ comparisons is said to be **$O(n)$**
- Said to have a linear run time
- Pronounced “on the order of n ” or simply “order n ”

O(n²) Algorithms

- Suppose you have an algorithm that tests whether any element of an array is duplicated elsewhere in the array
 - First element must be compared with every other element in the array
 - Second element must be compared with every other element except the first (it was already compared to the first)
 - The third element must be compared with every other element except the first two
- In the end, this algorithm makes $(n - 1) + (n - 2) + \dots + 2 + 1$ or $n^2/2 - n/2$ comparisons
- As n increases, the n^2 term dominates, and the n term becomes inconsequential
- Big O notation highlights the n^2 term, ignoring $n/2$

O(n²) Algorithms (cont.)

- Big O is concerned with how an algorithm's run time grows in relation to the number of items processed
- Suppose an algorithm requires n^2 comparisons
 - With four elements, the algorithm requires 16 comparisons
 - With eight elements, 64 comparisons
- Doubling the number of elements **quadruples** the number of comparisons
- Consider a similar algorithm requiring $n^2/2$ comparisons
 - With four elements, the algorithm requires eight comparisons; with eight elements, 32 comparisons
- Again, doubling the number of elements quadruples the number of comparisons
- Both algorithms grow as the square of n , so Big O ignores the constant, and both algorithms are considered to be $O(n^2)$
 - Referred to as quadratic run time and pronounced "on the order of n-squared" or "order n-squared."

Big O of the Linear Search

- Linear search algorithm runs in $O(n)$ time
- Worst case is that every element must be checked to determine whether the search item exists in the array
- If the size of the array is doubled, the number of comparisons that the algorithm must perform is also doubled
- Can provide outstanding performance if the element matching the search key happens to be at or near the front of the array
 - However, we seek algorithms that perform well, on average, across all searches
- Easy to program, but it can be slow compared to other search algorithms

11.9 Binary Search

The **binary search algorithm** is more efficient than linear search, but it requires a sorted array. The first iteration of this algorithm tests the middle element in the array. If this matches the search key, the algorithm ends. Assuming the array is sorted in ascending order, then if the search key is less than the middle element, it cannot match any element in the second half of the array and the algorithm continues with only the first half of the array (i.e., the first element up to, but not including, the middle element). If the search key is greater than the middle element, it cannot match any element in the first half of the array and the algorithm continues with only the second half (i.e., the element after the middle element through the last element). Each iteration tests the middle value of the remaining portion of the array. If the search key does not match the element, the algorithm eliminates half of the remaining elements. The algorithm ends by either finding an element that matches the search key or reducing the subarray to zero size.

Example

As an example consider the sorted 15-element array:

2 3 5 10 27 30 34 51 56 65 77 81 82 93 99

and a search key of 65. A program implementing the binary search algorithm would first check whether 51 is the search key (because 51 is the middle element of the array). The search key (65) is larger than 51, so 51 is ignored along with the first half of the array (all elements smaller than 51), leaving:

56 65 77 81 82 93 99

Next, the algorithm checks whether 81 (the middle element of the remainder of the array) matches the search key. The search key (65) is smaller than 81, so 81 is discarded along with the elements larger than 81. After just two tests, the algorithm has narrowed the number of values to check to only three (56, 65 and 77). It then checks 65 (which indeed matches the search key) and returns the index of the array element containing 65. This algorithm required just three comparisons to determine whether the search key matched an element of the array. Using a linear search algorithm would have required 10 comparisons. [Note: In this example, we used an array with 15 elements so that there's an obvious middle element. With an even number of elements, the middle of the array lies between two elements. We implement the algorithm to choose the higher of those two elements.]

11.9.1 Binary Search Implementation

Function `binary_search`

- Receives as parameters the array to search (`data`) and the search key (`key`)
- First calculates the low end index, high end index and middle index of the portion of the array that the program is currently searching
- Initially, the low end is 0, the high end is the length of the array minus 1 and the middle is the average of these two values
- Loops as long as low is less than or equal to high (the search is not complete) and location is equal to -1 (the key has not yet been found)
- If the value in the middle element is equal to the key, the loop terminates and the function returns `location` to the caller
- Each iteration of the loop eliminates half of the remaining values in the array

```

In [1]: # binarysearch.py
        """Use binary search to locate an item in an array."""
        import numpy as np

        def binary_search(data, key):
            """Perform binary search of data looking for key."""
            low = 0    # low end of search area
            high = len(data) - 1  # high end of search area
            middle = (low + high + 1) // 2  # middle element index
            location = -1  # return value -1 if not found

            # loop to search for element
            while low <= high and location == -1:
                # print remaining elements of array
                print(remaining_elements(data, low, high))

                print('    ' * middle, end='')  # output spaces for alignment
                print(' * ')  # indicate current middle

                # if the element is found at the middle
                if key == data[middle]:
                    location = middle  # location is the current middle
                elif key < data[middle]:  # middle element is too high
                    high = middle - 1  # eliminate the higher half
                else:  # middle element is too low
                    low = middle + 1  # eliminate the lower half

                middle = (low + high + 1) // 2  # recalculate the middle

            return location  # return location of search key

```

Function `remaining_elements`

- Shows the portion of the array being searched

```
In [2]: def remaining_elements(data, low, high):  
        """Display remaining elements of the binary search."""  
        return ' ' * low + ' '.join(str(s) for s in data[low:high + 1])
```

Function `main`

```
In [3]: def main():
        # create and display array of random values
        data = np.random.randint(10, 91, 15)
        data.sort()
        print(data, '\n')

        search_key = int(input('Enter an integer value (-1 to quit): '))

        # repeatedly input an integer; -1 terminates the program
        while search_key != -1:
            location = binary_search(data, search_key) # perform search

            if location == -1: # not found
                print(f'{search_key} was not found\n')
            else:
                print(f'{search_key} found in position {location}\n')

            search_key = int(input('Enter an integer value (-1 to quit): '))
```

```
In [4]: # call main to execute the search  
main()
```

```
[13 14 15 17 25 30 32 36 56 61 63 64 65 86 86]
```

```
13 14 15 17 25 30 32 36 56 61 63 64 65 86 86  
                        *
```

```
                    56 61 63 64 65 86 86  
                        *
```

```
                        65 86 86  
                            *
```

```
                            65  
                                *
```

```
65 found in position 12
```

```
13 14 15 17 25 30 32 36 56 61 63 64 65 86 86  
                        *
```

```
13 14 15 17 25 30 32  
                *
```

```
13 14 15  
    *
```

```
    15  
    *
```

```
16 was not found
```

11.9.2 Big O of the Binary Search

- Worst-case scenario: Searching a sorted array of 1023 elements takes only 10 comparisons with binary search
- Repeatedly dividing 1023 by 2 (because after each comparison we can eliminate half the array) and rounding down (because we also remove the middle element) yields 511, 255, 127, 63, 31, 15, 7, 3, 1 and 0
- 1023 (which is $2^{10} - 1$) is divided by 2 only 10 times to get the value 0, which indicates that there are no more elements to test
- Dividing by 2 is equivalent to one comparison in the binary search algorithm
- An array of 1,048,575 ($2^{20} - 1$) elements takes a maximum of 20 comparisons
- An array of about one billion elements takes a maximum of 30 comparisons
- Tremendous improvement over linear search
 - For a one-billion-element array, this is a difference between an average of 500 million comparisons for the linear search and a maximum of only 30 comparisons for the binary search!
- Maximum number of comparisons needed for the binary search of any sorted array is the exponent of the first power of 2 greater than the number of elements in the array, which is represented as $\log_2 n$
- From a Big O perspective, all logarithms grow at roughly the same rate, so in Big O, the base can be omitted
- $O(\log n)$ for a binary search, which is also known as logarithmic run time and pronounced as “order log n.”
- Assumes the array is sorted, which could take time

11.10 Sorting Algorithms

- Sorting data is one of the most important computing applications
- Virtually every organization must sort some data, and often massive amounts of it
- The end result—the sorted array—will be the same no matter which algorithm you use to sort the array
- Choice of algorithm affects only the run time and memory use of the program.

11.11 Selection Sort

- A simple, but inefficient, sorting algorithm
- First iteration selects the smallest element in the array and swaps it with the first element
- Second iteration selects the second-smallest item (which is the smallest item of the remaining elements) and swaps it with the second element
- The algorithm continues until the last iteration selects the second-largest element and swaps it with the second-to-last index, leaving the largest element in the last index

11.11 Selection Sort (cont.)

- Consider the array

34 56 14 20 77 51 93 30 15 52

- First iteration determines the smallest element (14) of this array (at index 2), then swaps 14 with 34, resulting in

14 56 34 20 77 51 93 30 15 52

- Second iteration determines the smallest value of the remaining elements (all elements except 14), which is 15 at index 8 and swaps 15 with 56, resulting in

14 15 34 20 77 51 93 30 56 52

- Third iteration determines the next smallest value (20) and swaps it with 34.

14 15 20 34 77 51 93 30 56 52

- Process continues until the array is fully sorted

14 15 20 30 34 51 52 56 77 93